

# Becoming a Robotics Engineer

A six-month self-study roadmap with curated resources, practice projects, and milestones

**SOURCE AND CREDIT**

This handout reorganizes and condenses the article **"How to become a Robotics Engineer in 6 months (RESOURCES)"** by **Ronin (@deronin\_)**, published on X on September 2, 2026. The roadmap, resource selection, and price research are the original author's work. Descriptions have been shortened and reworded for student use, so read the original for the full explanations.

**Original article:** [https://x.com/deronin\\_/status/2095180126359105955](https://x.com/deronin_/status/2095180126359105955)

**Prices and links:** checked by the original author in September 2026. Vendor prices change often, so confirm before you buy.

Prepared for Northeastern ECE robotics students by Prof. Xian (Shawn) Li.

## How to use this handout

**Self-study.** The plan assumes no prior electronics background and about two to three hours of work per day for six months, alongside your other commitments. Each month has a goal, curated resources, key concepts, practice tasks, and a milestone checklist. The practice tasks are the core of the plan, and the resources exist to support them.

**Coursework.** Use the matching month as background reading or as a refresher before a lab. The table below points to the sections most related to our robotics courses.

| Course                                    | Most relevant sections                                                                                                                                              |
|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| EECE 5550 Mobile Robotics                 | Month 4 (ROS 2, Gazebo, ros2_control, SLAM, Nav2); Month 5.1 and 5.2 (feedback control); Month 5.4 (perception); Month 2.4 and 2.5 (IMU filtering, balancing robot) |
| EECE 5554 Robotics Sensing and Navigation | Month 2.4 (sensors and filtering); Month 4 (ROS 2, SLAM, navigation); Month 5.4 (camera calibration and 3D perception)                                              |
| Assistive Robotics                        | Month 3.4 (robot arms); Month 5.3 and 5.5 (kinematics, MoveIt 2); Month 6 (learning from demonstration)                                                             |

**Documentation habit.** Keep every build in its own GitHub repository. Each README should include a photo or video, a wiring or architecture description, the numbers you measured, and a short section on what broke and how you fixed it. Reviewers and interviewers treat that last section as the clearest evidence of real engineering work.

**Budget.** Much of the plan can start at no cost. The circuit simulators, all of Month 4, and the Hugging Face robotics course need no hardware. Buy parts only when a practice task requires them, and check what the course labs already provide before you purchase anything.

## Roadmap at a glance

| Month | Focus                                 | Goal                                                   | Signature build                                                   |
|-------|---------------------------------------|--------------------------------------------------------|-------------------------------------------------------------------|
| 1     | Electronics, bench tools, Python, Git | Read a schematic, build a working circuit, find faults | Transistor switch, multimeter fault-finding, first GitHub repo    |
| 2     | Microcontrollers, motors, sensors     | A robot that moves, senses, and corrects itself        | Line follower (P, then PD) and a self-balancing robot             |
| 3     | CAD and fabrication                   | Design a part, make it, and have it fit                | Servo bracket, snap-fit enclosure, SO-101 arm with custom fingers |
| 4     | ROS 2 and simulation                  | Simulate a robot that maps and navigates on its own    | Custom xacro robot in Gazebo with SLAM Toolbox and Nav2           |
| 5     | Control, kinematics, perception       | Understand the math under the libraries                | PID comparison, LQR cart-pole, IK solver, MoveIt 2 pick and place |
| 6     | Robot learning and career             | Pick a direction, finish a portfolio piece, apply      | ACT policy on the SO-101 with measured success rates              |

## The field in brief

Robotics engineering is a group of specialties rather than one job. Common tracks include perception and computer vision, controls and state estimation, motion planning and manipulation, robot learning, embedded systems and firmware, simulation, systems integration, and deployment and operations. Most engineers go deep in one of these and stay conversant in the rest.

The original author reviewed job postings from robotics companies such as Figure and Skild and found four recurring expectations: working skill in both C++ and Python, experience on physical hardware rather than simulation alone, depth in one specialty with breadth across the others, and strong debugging ability. A specific degree was not on the list, which is why this roadmap centers on building things that work.

MONTH

1

# Electronics, the bench, and your core tools

**Goal:** Read a schematic, build a circuit that works, and track down the fault when it does not.

In robotics, many problems that look like software bugs are electrical: a loose wire, a sagging battery, or a motor pulling more current than the regulator can supply. You cannot diagnose these without understanding the electrical layer.

## 1.1 Electronics fundamentals

Start in a free simulator so mistakes cost nothing, and build every example yourself instead of only watching videos.

| Resource                                                                                                                                                                                             | Cost          | Use it for                                                             |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|------------------------------------------------------------------------|
| <b>Falstad Circuit Simulator</b><br><a href="http://www.falstad.com/circuit/">www.falstad.com/circuit/</a>                                                                                           | Free          | Browser simulator with animated current flow; builds intuition quickly |
| <b>Tinkercad Circuits (Autodesk)</b><br><a href="http://www.tinkercad.com/circuits">www.tinkercad.com/circuits</a>                                                                                   | Free, account | Virtual breadboard, Arduino, and multimeter in one place               |
| <b>All About Circuits: Lessons in Electric Circuits</b><br><a href="http://www.allaboutcircuits.com/textbook/">www.allaboutcircuits.com/textbook/</a>                                                | Free          | Open six-volume EE textbook for reference                              |
| <b>Afrotechmods Tutorials</b><br><a href="http://afrotechmods.com/tutorials/">afrotechmods.com/tutorials/</a>                                                                                        | Free          | Short, lively videos grouped by level                                  |
| <b>Make: Electronics, 3rd ed., Charles Platt</b><br><a href="http://www.makershed.com/products/make-electronics-3rd-edition-print">www.makershed.com/products/make-electronics-3rd-edition-print</a> | \$29.99       | Hands-on print book for first-time builders                            |

| Key concept                            | Explain                                                                    |
|----------------------------------------|----------------------------------------------------------------------------|
| <b>Ohm's law, voltage dividers</b>     | Practice until you can do them without thinking.                           |
| <b>Current draw</b>                    | A stalled motor can pull the supply down and reset your microcontroller.   |
| <b>Schematic symbols</b>               | Resistor, capacitor, diode, transistor, ground, and Vcc appear everywhere. |
| <b>Pull-up and pull-down resistors</b> | Needed for nearly every digital input you wire.                            |
| <b>Decoupling capacitors</b>           | Why each IC gets a small capacitor next to its power pin.                  |
| <b>LiPo battery basics</b>             | Cell count, C rating, and never leaving a LiPo charging unattended.        |

### PRACTICE TASK

Build a voltage divider in Falstad, predict the output by hand, and confirm it in the simulator. Then build a transistor switch that lights an LED from a logic-level input. This is the same circuit you will later use to let a 3.3 V pin control a load that needs more current than the pin can supply.

## 1.2 Bench tools and budget

| Tier | Approx. cost   | What it covers                                                                           |
|------|----------------|------------------------------------------------------------------------------------------|
| 0    | \$0            | Free simulators and the open textbook. Do this first.                                    |
| 1    | \$45 to \$60   | Solderless starter kit and a multimeter                                                  |
| 2    | \$110 to \$160 | Adds a soldering iron, solder, cutters, strippers, helping hands, perfboard, spare parts |
| 3    | \$200 to \$300 | Adds a bench power supply, better meter, desoldering pump, storage, robot chassis kit    |

| Resource                                                                                                                                                                                 | Cost               | Use it for                                              |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|---------------------------------------------------------|
| <b>Elegoo UNO R3 Super Starter Kit</b><br><a href="http://www.elegoo.com/products/elegoo-uno-r3-super-s-tarter-kit">www.elegoo.com/products/elegoo-uno-r3-super-s-tarter-kit</a>         | \$42.99            | Best value; the kit most beginner courses assume        |
| <b>Elegoo UNO Basic Starter Kit</b><br><a href="http://www.elegoo.com/products/elegoo-uno-basic-star-ter-kit">www.elegoo.com/products/elegoo-uno-basic-star-ter-kit</a>                  | \$19.99            | Lowest-cost real entry point                            |
| <b>SparkFun Inventor's Kit v4.1.2</b><br><a href="http://www.sparkfun.com/sparkfun-inventor-s-kit-v4-1-2.html">www.sparkfun.com/sparkfun-inventor-s-kit-v4-1-2.html</a>                  | \$99.95            | Strongest curriculum; its free guide helps with any kit |
| <b>Adafruit 9205B+ digital multimeter</b><br><a href="http://www.adafruit.com/product/2034">www.adafruit.com/product/2034</a>                                                            | \$17.50            | Voltage, current, continuity, resistance, capacitance   |
| <b>Pinecil V2 soldering iron</b><br><a href="http://pine64.com/product/pinecil-smart-mini-portabl-e-soldering-iron/">pine64.com/product/pinecil-smart-mini-portabl-e-soldering-iron/</a> | \$25.99 to \$35.99 | Temperature-controlled, USB-C powered, standard tips    |

| Vendor                       | When to use it                                                                            |
|------------------------------|-------------------------------------------------------------------------------------------|
| <b>AliExpress</b>            | Lowest prices by far, but 2 to 6 weeks of shipping and little help if a part arrives dead |
| <b>Amazon</b>                | Mid-priced and fast; a good choice for your first kit                                     |
| <b>Elegoo direct</b>         | Ships from regional warehouses in about a week                                            |
| <b>Adafruit, SparkFun</b>    | Higher prices, but every product links a tutorial and support is responsive               |
| <b>Seeed Studio, DFRobot</b> | Low to mid prices, worldwide shipping                                                     |
| <b>DigiKey, Mouser</b>       | Exact parts by specification with genuine datasheets                                      |
| <b>Pololu</b>                | Best source for motors and drivers; ships internationally                                 |

**PRACTICE TASK**

Get a kit and a multimeter, then measure. Check a battery's voltage, measure five resistors and compare them with their color bands, and use continuity mode to find a broken wire you made on purpose. This simple skill saves more hours than any other over the next six months.

## 1.3 Soldering

| Resource                                                                                                                                                                           | Cost | Use it for                                              |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|---------------------------------------------------------|
| <b>Adafruit Guide to Excellent Soldering</b><br><a href="http://learn.adafruit.com/adafruit-guide-excellent-s-oldering">learn.adafruit.com/adafruit-guide-excellent-s-oldering</a> | Free | Technique, photos of common failures, and safety        |
| <b>SparkFun: How to Use a Multimeter</b><br><a href="http://learn.sparkfun.com/tutorials/how-to-use-a-mul-timeter">learn.sparkfun.com/tutorials/how-to-use-a-mul-timeter</a>       | Free | All measurement modes, including replacing a blown fuse |

| Key concept       | Explain                                                                                  |
|-------------------|------------------------------------------------------------------------------------------|
| <b>Tip care</b>   | Keep the tip tinned and clean.                                                           |
| <b>Technique</b>  | Heat the joint, not the solder, and use flux; it fixes most problems blamed on the iron. |
| <b>Inspection</b> | Learn to recognize a cold joint by sight.                                                |
| <b>Order</b>      | Master through-hole parts before surface mount.                                          |

**PRACTICE TASK**

Solder header pins onto an inexpensive breakout board, check every pin with continuity mode, and repeat three times. Then desolder one part and resolder it, since poor removal is how most beginners damage boards.

**1.4 Python, the terminal, and Git**

| Resource                                                                                                                                        | Cost          | Use it for                                            |
|-------------------------------------------------------------------------------------------------------------------------------------------------|---------------|-------------------------------------------------------|
| <b>CS50P: Introduction to Programming with Python (Harvard)</b><br><a href="https://cs50.harvard.edu/python/">cs50.harvard.edu/python/</a>      | Free          | Rigorous, with problem sets and a final project       |
| <b>Python for Everybody (Coursera)</b><br><a href="https://www.coursera.org/specializations/python">www.coursera.org/specializations/python</a> | Free to audit | A gentler alternative if CS50P feels steep            |
| <b>The Missing Semester of Your CS Education (MIT)</b><br><a href="https://missing.csail.mit.edu/">missing.csail.mit.edu/</a>                   | Free          | Shell, scripting, and command-line fluency            |
| <b>Learn Git Branching</b><br><a href="https://learngitbranching.js.org/">learngitbranching.js.org/</a>                                         | Free          | Interactive, visual practice with branches and merges |

| Key concept     | Explain                                                                   |
|-----------------|---------------------------------------------------------------------------|
| <b>Python</b>   | Functions, classes, file I/O, JSON, virtual environments, pip             |
| <b>Terminal</b> | cd, ls, grep, running scripts, environment variables, ssh                 |
| <b>Git</b>      | init, add, commit, push, branches, and a README another person can follow |

**PRACTICE TASK**

From now on, every project gets its own GitHub repository with a photo, wiring notes, and a short account of what failed and how you fixed it.

**MONTH 1 MILESTONE: BY THE END OF THIS MONTH YOU CAN**

- ☐ Build a breadboard circuit from its schematic
- ☐ Confirm a resistor value by calculation before powering up
- ☐ Locate a short, an open, or a dead part with a multimeter
- ☐ Make a clean through-hole joint and verify it electrically
- ☐ Write a Python script, run it from the terminal, and push it to GitHub
- ☐ Explain why a stalling motor can reset a microcontroller

MONTH

2

# Microcontrollers, motors, sensors, and your first moving robot

**Goal:** Build a robot that moves, senses its surroundings, and corrects its own behavior.

A line-following robot already contains the whole pattern of robotics: sensor input, actuator output, a closed loop, and a tuning problem. Larger robots follow the same pattern at higher cost.

## 2.1 Arduino

Begin with Arduino for its huge ecosystem, then move to the ESP32 within a few weeks. Everything carries over.

| Resource                                                                                                                        | Cost | Use it for                                               |
|---------------------------------------------------------------------------------------------------------------------------------|------|----------------------------------------------------------|
| <b>Paul McWhorter, Arduino Lessons</b><br><a href="http://toptechboy.com/arduino-lessons/">toptechboy.com/arduino-lessons/</a>  | Free | 100+ slow-paced lessons with homework                    |
| <b>Arduino Built-in Examples</b><br><a href="https://docs.arduino.cc/built-in-examples/">docs.arduino.cc/built-in-examples/</a> | Free | Ready-to-run sketches bundled with the IDE               |
| <b>Arduino Docs, Learn section</b><br><a href="https://docs.arduino.cc/learn/">docs.arduino.cc/learn/</a>                       | Free | Reference for digital and analog IO, PWM, I2C, SPI, UART |
| <b>Arduino Project Hub</b><br><a href="https://projecthub.arduino.cc/">projecthub.arduino.cc/</a>                               | Free | Thousands of projects with wiring and code               |

| Key concept                | Explain                                                                    |
|----------------------------|----------------------------------------------------------------------------|
| <b>IO and PWM</b>          | digitalWrite, digitalRead, analogRead, analogWrite, and what PWM really is |
| <b>Interrupts</b>          | Why polling a button inside the loop eventually breaks                     |
| <b>I2C and SPI</b>         | Wiring each bus and finding a sensor's address in its datasheet            |
| <b>Serial debugging</b>    | Your main debugging tool for months                                        |
| <b>Non-blocking timing</b> | Use millis() instead of delay(); delay() causes problems in every robot    |

### PRACTICE TASK

Build a reaction-timer game: an LED turns on after a random wait, a button press stops the timer, and the time in milliseconds prints to serial. It exercises interrupts, debouncing, and non-blocking timing.

## 2.2 ESP32

The ESP32 adds WiFi, Bluetooth, more processing power, and two cores, and it costs less than an Uno. Use an ESP32-S3 as your main board and keep one classic ESP32 so older tutorial code runs unchanged.

| Resource                                                                                                                                                                                                                    | Cost | Use it for                                                    |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|---------------------------------------------------------------|
| <b>Random Nerd Tutorials: Getting Started with ESP32</b><br><a href="https://randomnerdtutorials.com/getting-started-with-esp32/">randomnerdtutorials.com/getting-started-with-esp32/</a>                                   | Free | Large project library with fixes for common problems          |
| <b>ESP-IDF Programming Guide (Espressif)</b><br><a href="https://docs.espressif.com/projects/esp-idf/en/stable/esp32/get-started/index.html">docs.espressif.com/projects/esp-idf/en/stable/esp32/get-started/index.html</a> | Free | Official native toolchain and build system                    |
| <b>Arduino ESP32 Core documentation (Espressif)</b><br><a href="https://docs.espressif.com/projects/arduino-esp32/en/latest/">docs.espressif.com/projects/arduino-esp32/en/latest/</a>                                      | Free | Official docs for the Arduino layer                           |
| <b>DroneBot Workshop ESP32 hub</b><br><a href="https://dronebotworkshop.com/esp32-2/">dronebotworkshop.com/esp32-2/</a>                                                                                                     | Free | Detailed articles and videos on ESP-NOW, OTA, low-power modes |

| Resource                                                                                                                                                 | Cost             | Use it for                        |
|----------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|-----------------------------------|
| <b>ESP32-S3-DevKitC-1, 8MB (Adafruit)</b><br><a href="http://www.adafruit.com/product/5312">www.adafruit.com/product/5312</a>                            | \$15.95          | Recommended main board            |
| <b>Classic ESP32 Dev Board (Adafruit)</b><br><a href="http://www.adafruit.com/product/3269">www.adafruit.com/product/3269</a>                            | \$15.00          | Runs older tutorial code          |
| <b>Seeed XIAO ESP32-C3</b><br><a href="http://www.seeedstudio.com/Seeed-XIAO-ESP32C3-p-5431.html">www.seeedstudio.com/Seeed-XIAO-ESP32C3-p-5431.html</a> | \$4.99           | When space is tight               |
| <b>Generic ESP32 clones (AliExpress)</b>                                                                                                                 | About \$4 to \$9 | Common street price; not verified |

| Framework                | Choose it for                                                                                       |
|--------------------------|-----------------------------------------------------------------------------------------------------|
| <b>Arduino framework</b> | Fastest path to a working robot, with the largest library ecosystem                                 |
| <b>ESP-IDF</b>           | Full control over tasks, cores, power, and timing                                                   |
| <b>MicroPython</b>       | Quick sensor experiments; avoid for balancing loops, since garbage-collection pauses disrupt timing |

**PRACTICE TASK**

Serve a web page from the ESP32 that shows live sensor readings and has buttons that move a servo, then open it from your phone on the same network. This covers WiFi, HTTP handling, and asynchronous code together.

## 2.3 Motors, drivers, and actuation

| Motor type                    | Explain                                                                                              |
|-------------------------------|------------------------------------------------------------------------------------------------------|
| <b>Brushed DC gearmotor</b>   | Low cost, needs an H-bridge, no position feedback without an encoder. The default for a first rover. |
| <b>Hobby servo</b>            | Internal position loop with about 180 degrees of travel and no feedback output                       |
| <b>Smart serial bus servo</b> | Daisy-chained, reports position, velocity, and current; used in modern low-cost arms                 |
| <b>Stepper</b>                | Open-loop absolute positioning with high holding torque                                              |

| Resource                                                                                                                                                                               | Cost         | Use it for                                                        |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|-------------------------------------------------------------------|
| <b>DroneBot Workshop: DC Motors with the L298N</b><br><a href="http://dronebotworkshop.com/dc-motors-l298n-h-bridge/">dronebotworkshop.com/dc-motors-l298n-h-bridge/</a>               | Free         | PWM and H-bridge basics, ending in a robot car                    |
| <b>SparkFun TB6612FNG Hookup Guide</b><br><a href="http://learn.sparkfun.com/tutorials/tb6612fng-hookup-guide/all">learn.sparkfun.com/tutorials/tb6612fng-hookup-guide/all</a>         | Free         | Wiring and library for the driver you should actually use         |
| <b>DroneBot Workshop: Stepper Motors with Arduino</b><br><a href="http://dronebotworkshop.com/stepper-motors-with-arduino/">dronebotworkshop.com/stepper-motors-with-arduino/</a>      | Free         | Driver types, microstepping, NEMA sizing                          |
| <b>SimpleFOC documentation</b><br><a href="http://docs.simplefoc.com/">docs.simplefoc.com/</a>                                                                                         | Free         | Clear introduction to field-oriented control for brushless motors |
| <b>Adafruit DRV8833 motor driver</b><br><a href="http://www.adafruit.com/product/3297">www.adafruit.com/product/3297</a>                                                               | \$5.95       | Lowest-cost good driver                                           |
| <b>SparkFun TB6612FNG breakout</b><br><a href="http://www.sparkfun.com/sparkfun-motor-driver-dual-tb6612fng-1a.html">www.sparkfun.com/sparkfun-motor-driver-dual-tb6612fng-1a.html</a> | \$14.77      | Recommended replacement for the L298N                             |
| <b>Pololu gearmotor with encoder</b><br><a href="http://www.pololu.com/product/3675">www.pololu.com/product/3675</a>                                                                   | \$19.95 each | Encoder wiring already handled                                    |

| Resource                                                                                                                                                                                                                                 | Cost    | Use it for                       |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|----------------------------------|
| <b>Pololu A4988 stepper driver carrier</b><br><a href="http://www.pololu.com/product/1182">www.pololu.com/product/1182</a>                                                                                                               | \$8.95  | Standard stepper driver          |
| <b>Feetech STS3215 smart servo, 12 V (RobotShop)</b><br><a href="http://www.robotshop.com/products/feetech-12v-30kgcm-magnetic-encoding-servo-sts3215">www.robotshop.com/products/feetech-12v-30kgcm-magnetic-encoding-servo-sts3215</a> | \$31.71 | The servo used in the SO-101 arm |

**AVOID THE L298N**

The L298N appears in most tutorials but is outdated. It loses about 2 V in its output stage, runs hot, and drains your battery. Learn from those tutorials, then build with a TB6612FNG or DRV8833.

**PRACTICE TASK**

Run a DC motor forward and backward at five PWM speeds. Then add an encoder and write a function that turns the wheel exactly one revolution regardless of battery voltage. That second part is your first real closed loop, and it is harder than it looks.

## 2.4 Sensors and state estimation

| Resource                                                                                                                                                                                                                                       | Cost | Use it for                                              |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|---------------------------------------------------------|
| <b>Adafruit BNO085 9-DoF IMU guide</b><br><a href="http://learn.adafruit.com/adafruit-9-dof-orientation-imu-fusion-breakout-bno085/overview">learn.adafruit.com/adafruit-9-dof-orientation-imu-fusion-breakout-bno085/overview</a>             | Free | IMU with on-chip fusion that outputs a quaternion       |
| <b>Kalman and Bayesian Filters in Python, Roger Labbe</b><br><a href="http://rlabbe.github.io/Kalman-and-Bayesian-Filters-in-Python/">rlabbe.github.io/Kalman-and-Bayesian-Filters-in-Python/</a>                                              | Free | Runnable notebooks from g-h filters to particle filters |
| <b>MathWorks: Understanding Sensor Fusion and Tracking</b><br><a href="http://www.mathworks.com/videos/series/understanding-sensor-fusion-and-tracking.html">www.mathworks.com/videos/series/understanding-sensor-fusion-and-tracking.html</a> | Free | Six-part conceptual video series                        |

| Sensor                                       | Price   | Notes                                                         |
|----------------------------------------------|---------|---------------------------------------------------------------|
| <b>HC-SR04 ultrasonic</b>                    | \$3.95  | Cheap obstacle detection; wide cone, weak on soft surfaces    |
| <b>VL53L0X time-of-flight</b>                | \$14.95 | Narrow 35-degree cone, more reliable ranging                  |
| <b>MPU-6050 6-DoF IMU</b>                    | \$12.95 | You implement the fusion yourself, which is the learning goal |
| <b>BNO085 9-DoF IMU</b>                      | \$29.50 | Fusion on-chip, with a UART mode suited to robots             |
| <b>Pololu magnetic encoder pair</b>          | \$8.95  | Adds odometry to motors without encoders                      |
| <b>RPLIDAR C1 360-degree lidar (DFRobot)</b> | \$69.00 | Newer, lower-cost option than the classic A1                  |

**TIP**

For a balancing robot, start with a complementary filter:  $\text{angle} = a * (\text{angle} + \text{gyro} * \text{dt}) + (1 - a) * \text{accelAngle}$ , with a around 0.98. Move to a Kalman filter once you understand where the complementary filter falls short.

**PRACTICE TASK**

Mount an IMU on a board, print the pitch angle to serial, and hold it still. The value drifts. Add a complementary filter and watch the drift disappear. This is the central lesson of state estimation in about twenty minutes.

## 2.5 Your first two robots

| Build                   | Approx. cost | Bill of materials                                                                                                                                                    |
|-------------------------|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Line follower, quality  | \$105        | ESP32-S3 \$15.95, Pololu Romi chassis kit \$39.95, TB6612FNG \$14.77, QTR-8RC reflectance array \$12.95, batteries and holder ~\$12, wiring and headers ~\$10        |
| Line follower, budget   | \$38         | Generic ESP32 ~\$6, 2WD acrylic chassis ~\$12, DRV8833 \$5.95, five TCRT5000 sensors ~\$3, batteries ~\$6, wiring ~\$5                                               |
| Self-balancing, quality | \$134        | ESP32 \$15.95, two encoder gearmotors \$39.90, TB6612FNG \$14.77, MPU-6050 \$12.95, printed or laser-cut chassis ~\$10, LiPo, charger, and wheels ~\$30, misc. ~\$10 |
| Self-balancing, budget  | \$62         | Lower-cost equivalents of the parts above                                                                                                                            |

### PRACTICE TASKS

Build the line follower first and tune it with a P controller. Add the D term and watch the oscillation go away, filming both versions. A side-by-side video of the untuned and tuned robot shows that you understand the loop rather than copied a gain.

Then build the balancer. It will not stand until both the filter and the loop timing are right, and working through that is the point of the exercise.

### MONTH 2 MILESTONE: BY THE END OF THIS MONTH YOU CAN

- ☐ Drive a motor at a controlled speed and explain PWM duty cycle versus actual RPM
- ☐ Read an encoder and close a position loop around it
- ☐ Wire and read an I2C sensor using only its datasheet
- ☐ Fuse accelerometer and gyroscope data into a stable angle estimate
- ☐ Explain P, I, and D by describing what your robot did when you changed each
- ☐ Publish two working robots on GitHub with wiring, code, and notes on what broke

MONTH

3

## Mechanical design, CAD, and making your own parts

**Goal:** Design a part in CAD, manufacture it, and have it fit.

Designing your own parts is the step from assembling kits to building robots. Once you can go from an idea to a physical bracket in an afternoon, far more projects become possible.

### 3.1 CAD

Choose one tool and learn it well rather than trying several.

| Tool                  | Cost                        | Best for                                                                   | Watch out for                                                                 |
|-----------------------|-----------------------------|----------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Onshape Free          | Free                        | Runs in a browser on any machine; assemblies behave like real robot joints | Every document is public on the free tier                                     |
| Fusion Personal       | Free, renewable 3-year term | Later CAM and 3D-print workflows                                           | Non-commercial use under \$1,000/year revenue; limited file import and export |
| FreeCAD               | Free, open source           | No cloud account or payment needed; version 1.1 (March 2026) is solid      | None noted by the original author                                             |
| SOLIDWORKS for Makers | \$48/year                   | The industry-standard tool                                                 | Files are watermarked and will not open in commercial SOLIDWORKS              |

| Resource                                                                                                                                                                                                                                            | Cost          | Use it for                                                       |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|------------------------------------------------------------------|
| <b>Onshape Learning Center: Fundamentals, CAD</b><br><a href="https://learn.onshape.com/learning-paths/onshape-fundamentals-cad">learn.onshape.com/learning-paths/onshape-fundamentals-cad</a>                                                      | Free, account | Structured path with a credential and a robotics track           |
| <b>Product Design Online: Learn Autodesk Fusion in 30 Days</b><br><a href="https://productdesignonline.com/learn-autodesk-fusion-360-in-30-days-official-course/">productdesignonline.com/learn-autodesk-fusion-360-in-30-days-official-course/</a> | Free          | One modeled object per day                                       |
| <b>MangoJelly Solutions (FreeCAD)</b><br><a href="https://www.youtube.com/@MangoJellySolutions">www.youtube.com/@MangoJellySolutions</a>                                                                                                            | Free          | Short, focused FreeCAD lessons                                   |
| <b>Protolabs Network: Design for 3D printing</b><br><a href="https://www.hubs.com/knowledge-base/design-for-3d-printing/">www.hubs.com/knowledge-base/design-for-3d-printing/</a>                                                                   | Free          | Wall thickness, orientation, tolerances, snap-fits, file formats |

| Key concept                       | Explain                                                              |
|-----------------------------------|----------------------------------------------------------------------|
| <b>Fully constrained sketches</b> | Under-constrained sketches shift when you edit them later            |
| <b>Parametric design</b>          | Drive dimensions from variables so one change updates the whole part |
| <b>Assemblies and mates</b>       | Revolute, slider, and fixed mates that match real joints             |
| <b>Design from datasheets</b>     | Start from the real dimensions of hardware you own                   |
| <b>Export formats</b>             | STEP for sharing designs, STL for printing                           |

#### PRACTICE TASK

Model a bracket for the exact servo you own, with correctly sized screw holes and shaft clearance, using the manufacturer's datasheet drawing. Print it and test the fit. It will probably not fit the first time, and that is the lesson.

## 3.2 3D printing

| Resource                                                                                                                                                                                        | Cost     | Use it for                                    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|-----------------------------------------------|
| <b>Creality Ender-3 V3 SE</b><br><a href="https://store.creality.com/products/ender-3-v3-se-3d-printer">store.creality.com/products/ender-3-v3-se-3d-printer</a>                                | \$199    | Lowest-cost printer on the list               |
| <b>Bambu Lab A1 mini</b><br><a href="https://www.bestbuy.com/product/bambu-lab-a1-mini-3d-printer-silver/CZTZV9ZGGV">www.bestbuy.com/product/bambu-lab-a1-mini-3d-printer-silver/CZTZV9ZGGV</a> | \$219.99 | Compact printer                               |
| <b>Bambu Lab A1</b><br><a href="https://www.bestbuy.com/product/bambu-lab-a1-3d-printer-silver/CZW2ZH33H4">www.bestbuy.com/product/bambu-lab-a1-3d-printer-silver/CZW2ZH33H4</a>                | \$299.99 | 256 mm bed for larger brackets                |
| <b>Creality K1C</b><br><a href="https://store.creality.com/products/k1c-3d-printer">store.creality.com/products/k1c-3d-printer</a>                                                              | \$369    | Enclosed, hardened for carbon-fiber filaments |
| <b>Bambu Lab P1S</b><br><a href="https://us.store.bambulab.com/products/p1s">us.store.bambulab.com/products/p1s</a>                                                                             | \$799    | Enclosed CoreXY for ABS and ASA               |

| Material                   | Use it for                                                             | Notes                                                                   |
|----------------------------|------------------------------------------------------------------------|-------------------------------------------------------------------------|
| <b>PLA, PLA+</b>           | Prototypes, jigs, and the SO-101 arm (PLA+, 15% infill, 0.2 mm layers) | Stiff and easy; creeps under steady load and softens around 55 to 60 °C |
| <b>PETG</b>                | Default functional parts: chassis plates, housings, servo mounts       | Tough with strong layer adhesion; tends to string                       |
| <b>ABS, ASA</b>            | Parts near hot motors and outdoor robots                               | Warp badly without an enclosure                                         |
| <b>Nylon</b>               | Gears and cable guides                                                 | Often easier to order than to print                                     |
| <b>Carbon-fiber filled</b> | Stiff structural links                                                 | Abrasive; needs a hardened nozzle                                       |
| <b>TPU</b>                 | Feet, bumpers, compliant gripper fingers                               | Flexible                                                                |

| Resource                                                                                                                                                                                                                 | Cost     | Use it for                                                     |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|----------------------------------------------------------------|
| <b>OrcaSlicer Calibration wiki</b><br><a href="https://github.com/OrcaSlicer/OrcaSlicer/wiki/Calibration">github.com/OrcaSlicer/OrcaSlicer/wiki/Calibration</a>                                                          | Free     | Calibration steps in a recommended order                       |
| <b>Teaching Tech 3D Printer Calibration</b><br><a href="https://teachingtecht.github.io/calibration.html">teachingtecht.github.io/calibration.html</a>                                                                   | Free     | Interactive walkthrough for any printer                        |
| <b>CNC Kitchen</b><br><a href="https://www.youtube.com/@CNCKitchen">www.youtube.com/@CNCKitchen</a>                                                                                                                      | Free     | Measured strength tests on infill, walls, inserts, orientation |
| <b>Clearance and Tolerance 3D Printer Gauge</b><br><a href="https://www.printables.com/model/57067-clearance-and-tolerance-3d-printer-gauge">www.printables.com/model/57067-clearance-and-tolerance-3d-printer-gauge</a> | Free STL | Measures your printer's real fit clearances                    |

**No printer?** Use a Fab Lab (about 2,875 worldwide, [fablabs.io/labs](https://fablabs.io/labs)), a public library makerspace ([action.everylibrary.org/how\\_to\\_find\\_a\\_makerspace\\_near\\_you](https://action.everylibrary.org/how_to_find_a_makerspace_near_you)), Craftcloud for quotes across 95 countries ([craftcloud3d.com/](https://craftcloud3d.com/)), or JLC3DP, from \$1.00 per part with about 3-day builds ([jlc3dp.com/](https://jlc3dp.com/)). Check campus fabrication spaces first.

**Cost check.** The SO-101 needs roughly 1 kg of PLA+, about \$20 to \$25 of filament, compared with \$30.99 for a ready-printed set. A printer does not pay for itself on one build. It pays off through iteration, since the tenth revision of a gripper finger costs cents and minutes at home.

**PRACTICE TASK**

Print the tolerance gauge and record your printer's actual clearances. Then design and print a two-part snap-fit enclosure for your ESP32 that closes without glue. Iterate until it clicks. This design, print, test loop is what all mechanical design is made of.

### 3.3 Actuators and transmissions

| Key concept           | Explain                                                                     |
|-----------------------|-----------------------------------------------------------------------------|
| <b>Gear ratios</b>    | The trade between speed and torque                                          |
| <b>Backlash</b>       | Reduces position accuracy in a way software cannot fully correct            |
| <b>Bearings</b>       | Selection and preload                                                       |
| <b>Drive type</b>     | Belt versus gear versus direct drive                                        |
| <b>Failure points</b> | A cheap servo's plastic gears are usually the first thing to fail on an arm |

**PRACTICE TASK**

Design and print a simple planetary or cycloidal reducer for a NEMA 17 stepper or a hobby motor. Measure backlash by holding the output and rocking it, then redesign to reduce it. Reference build:

[www.instructables.com/OpenCycloid-3D-printed-Open-Source-Robotic-Actuator/](http://www.instructables.com/OpenCycloid-3D-printed-Open-Source-Robotic-Actuator/)

### 3.4 Build a real robot arm: the SO-101

The SO-101 is an open-source 5-DOF arm with a gripper from TheRobotStudio and Hugging Face. It is built as a leader and follower pair: you move the leader by hand and the follower mirrors it. That teleoperation setup is how you record the demonstrations used in Month 6. The official bill of materials lists \$229.88 for a pair and \$121.94 for one follower arm, excluding printing ([github.com/TheRobotStudio/SO-ARM100](https://github.com/TheRobotStudio/SO-ARM100)).

| Resource                                                                                                                                                                                                | Cost                 | Use it for                                                  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|-------------------------------------------------------------|
| <b>Seeed Studio SO-ARM101 Pro servo kit</b><br><a href="http://www.seeedstudio.com/SO-ARM101-Low-Cost-AI-Arm-Kit-Pro-p-6427.html">www.seeedstudio.com/SO-ARM101-Low-Cost-AI-Arm-Kit-Pro-p-6427.html</a> | \$277.99             | Motors and control boards, no printed parts                 |
| <b>Seeed Studio printed parts set</b><br><a href="http://www.seeedstudio.com/SO-ARM101-3D-printed-Enclosure-p-6428.html">www.seeedstudio.com/SO-ARM101-3D-printed-Enclosure-p-6428.html</a>             | \$30.99              | If you have no printer                                      |
| <b>Robonine SO-ARM101 complete kit</b><br><a href="http://robonine.com/shop/so-arm101-black-robotic-arm-kit/">robonine.com/shop/so-arm101-black-robotic-arm-kit/</a>                                    | \$349.00             | Ships from Delaware                                         |
| <b>WowRobo via OpenELAB</b><br><a href="http://openelab.com/products/wowrobo-robotics-so-arm101-diykit">openelab.com/products/wowrobo-robotics-so-arm101-diykit</a>                                     | \$325.99 to \$489.99 | Parts plus servos, unassembled full kit, or fully assembled |

| Budget              | Lower-cost alternative                                                                                                                                                              |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>\$0</b>          | Run the full LeRobot software stack in MuJoCo simulation before any hardware exists                                                                                                 |
| <b>\$50 to \$80</b> | EEZYbotARM MK2 with free STLs and MG996R servos; teaches linkage kinematics ( <a href="http://www.thingiverse.com/thing:1454048">www.thingiverse.com/thing:1454048</a> )            |
| <b>\$122</b>        | One SO-101 follower arm with self-printed parts; no teleoperation, full software path                                                                                               |
| <b>\$199.99</b>     | Hiwonder xArm 1S, the lowest-cost arm with bus servos that report position and voltage ( <a href="http://www.hiwonder.com/products/xarm-1s">www.hiwonder.com/products/xarm-1s</a> ) |

**PRACTICE TASK**

Build the SO-101, calibrate every servo, and teleoperate the follower with the leader. Then design and print your own TPU gripper fingers and test them on three objects with different shapes. The arm supports Months 5 and 6, and the custom fingers show you can design as well as assemble.

**MONTH 3 MILESTONE: BY THE END OF THIS MONTH YOU CAN**

- ☐ Model a part from a datasheet using fully constrained sketches
- ☐ State your printer's measured clearance values
- ☐ Design for FDM with orientation, overhangs, and layer adhesion in mind
- ☐ Choose PLA, PETG, ABS, or TPU for a given part and justify the choice
- ☐ Explain backlash and demonstrate it on something you built
- ☐ Show a working arm you assembled, calibrated, and modified with your own parts

MONTH

4

## ROS 2, simulation, and building robots the industry way

**Goal:** Build a robot in ROS 2, simulate it, and have it map a room and navigate on its own.

ROS 2 is the industry standard and one of the most requested skills in robotics job postings. Everything this month runs on an ordinary laptop with no robot and no NVIDIA GPU.

### 4.1 Choosing a ROS 2 distribution

ROS 2 releases each May. Even-year releases are LTS with five years of support, and odd-year releases get about eighteen months. Always install the Gazebo version paired with your ROS 2 distribution.

| Distribution     | Status (Sept. 2026)   | End of life   | Ubuntu | Gazebo   | Advice                            |
|------------------|-----------------------|---------------|--------|----------|-----------------------------------|
| Lyrical Luth     | Current LTS, May 2026 | May 2031      | 26.04  | Jetty    | Move here once tutorials catch up |
| Jazzy Jalisco    | LTS, May 2024         | May 2029      | 24.04  | Harmonic | <b>Recommended starting point</b> |
| Kilted Kaiju     | Non-LTS               | Dec. 31, 2026 | 24.04  | Ionic    | Do not start here                 |
| Humble Hawksbill | Legacy LTS            | May 2027      | 22.04  | Fortress | Legacy only                       |

#### RECOGNIZE ROS 1 CONTENT

ROS 1 Noetic reached end of life on May 31, 2025, with no successor, yet much highly ranked tutorial content is still ROS 1. If you see `catkin_make`, `roscore`, `roslaunch`, `rospy`, or XML-only launch files, find a different tutorial. ROS 2 uses `colcon build`, has no master, and uses `ros2 run` and Python launch files.

### 4.2 ROS 2 core concepts

| Resource                                                                                                                                                               | Cost                              | Use it for                                                                       |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------|----------------------------------------------------------------------------------|
| <b>Official ROS 2 Tutorials (Jazzy)</b><br><a href="https://docs.ros.org/en/jazzy/Tutorials.html">docs.ros.org/en/jazzy/Tutorials.html</a>                             | Free                              | Canonical reference; pair it with videos                                         |
| <b>The Construct</b><br><a href="https://www.theconstruct.ai/">www.theconstruct.ai/</a>                                                                                | Free tier; paid from €39.97/month | Browser-based ROS environment, no install needed                                 |
| <b>Edouard Renard: ROS 2 for Beginners, Level 1 (Udemy)</b><br><a href="https://www.udemy.com/course/ros2-for-beginners/">www.udemy.com/course/ros2-for-beginners/</a> | About \$10 to \$20 on sale        | 13 hours in Python and C++; never pay list price                                 |
| <b>Articulated Robotics (Josh Newans)</b><br><a href="https://articulatedrobotics.xyz/tutorials/">articulatedrobotics.xyz/tutorials/</a>                               | Free                              | End to end: URDF, simulation, <code>ros2_control</code> , real robot, SLAM, Nav2 |
| <b>MOGI-ROS university course</b><br><a href="https://github.com/orgs/MOGI-ROS/repositories">github.com/orgs/MOGI-ROS/repositories</a>                                 | Free                              | Full semester on Jazzy and Gazebo Harmonic, with code each week                  |
| <b>Automatic Addison</b><br><a href="https://automaticaddison.com/tutorials/">automaticaddison.com/tutorials/</a>                                                      | Free                              | Recipe-style how-tos, including Lyrical tracks                                   |

| Key concept                   | Explain                                                             |
|-------------------------------|---------------------------------------------------------------------|
| <b>Communication patterns</b> | Nodes, topics, services, and actions, and which one a problem needs |
| <b>Interfaces</b>             | Custom message and service definitions                              |
| <b>Configuration</b>          | Parameters and YAML files                                           |
| <b>Launch files</b>           | Python launch files with arguments and topic remapping              |
| <b>Workspaces</b>             | <code>colcon</code> workspaces and package layout                   |

| Key concept     | Explain                                                                      |
|-----------------|------------------------------------------------------------------------------|
| <b>ros2 bag</b> | Recording and replaying data to debug problems that happen only occasionally |

**KNOWN GAP**

DDS and QoS settings are poorly covered by tutorials. If a topic publishes but nothing receives it, check for a QoS mismatch first, and use the official ROS 2 Concepts pages.

**PRACTICE TASK**

Build a multi-node system with no robot or simulator: a sensor publisher, a processing node, a service-based configuration node, and a parameterized aggregator, with your own .msg and .srv, all started from a Python launch file that takes arguments. Record it with ros2 bag and replay it.

### 4.3 URDF, TF, and describing a robot

| Resource                                                                                                                                                                                                                                                                             | Cost                       | Use it for                                         |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|----------------------------------------------------|
| <b>Articulated Robotics: Coordinate Transforms for Robotics</b><br><a href="https://articulatedrobotics.xyz/category/coordinate-transforms-for-robotics">articulatedrobotics.xyz/category/coordinate-transforms-for-robotics</a>                                                     | Free                       | Frames and transforms, the usual URDF blocker      |
| <b>Edouard Renard: Level 2, TF, URDF, RViz, Gazebo (Udemy)</b><br><a href="https://www.udemy.com/course/ros2-tf-urdf-rviz-gazebo/">www.udemy.com/course/ros2-tf-urdf-rviz-gazebo/</a>                                                                                                | About \$10 to \$20 on sale | xacro, robot_state_publisher, RViz, Gazebo plugins |
| <b>Official tutorial: URDF with robot_state_publisher</b><br><a href="https://docs.ros.org/en/jazzy/Tutorials/Intermediate/URDF/Using-URDF-with-Robot-State-Publisher-cpp.html">docs.ros.org/en/jazzy/Tutorials/Intermediate/URDF/Using-URDF-with-Robot-State-Publisher-cpp.html</a> | Free                       | Canonical walkthrough in C++ and Python            |

| Common issue                                                     | Fix                                                                          |
|------------------------------------------------------------------|------------------------------------------------------------------------------|
| <b>Mixing up joint_state_publisher and robot_state_publisher</b> | The first supplies joint positions; the second computes transforms from them |
| <b>Missing inertial tags</b>                                     | The robot explodes or sinks in Gazebo; always set mass and inertia           |
| <b>No static transform to the lidar frame</b>                    | A leading cause of beginner SLAM failures                                    |
| <b>Hand-writing hundreds of lines of XML</b>                     | Use xacro macros from the start                                              |

**PRACTICE TASK**

Describe your own robot in xacro rather than using TurtleBot: a differential-drive base, a sensor mast, and a two-DOF pan-tilt head, with correct inertias and separate collision and visual geometry. Move the joints with joint\_state\_publisher\_gui and view the full TF tree in RViz.

### 4.4 Simulation

Gazebo Classic (versions 1 to 11) reached end of life in January 2025. Its rewrite was briefly called Ignition, then renamed Gazebo in 2022, and every ign command became gz. A tutorial using ros launch gazebo\_ros or ign gazebo is out of date. Current releases, in order: Fortress, Garden, Harmonic, Ionic, Jetty.

| Resource                                                                                                                                 | Cost              | Use it for                                                             |
|------------------------------------------------------------------------------------------------------------------------------------------|-------------------|------------------------------------------------------------------------|
| <b>Gazebo documentation and tutorials</b><br><a href="https://gazebo.org/docs/latest/getstarted/">gazebo.org/docs/latest/getstarted/</a> | Free              | Building and moving robots, SDF worlds, sensors, spawning URDF         |
| <b>MuJoCo</b><br><a href="https://mujoco.readthedocs.io/en/stable/overview.html">mujoco.readthedocs.io/en/stable/overview.html</a>       | Free, open source | Fast, accurate contact dynamics on CPU; research standard for learning |

| Resource                                                                                                                                                                                         | Cost                                                                                | Use it for                                                       |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|------------------------------------------------------------------|
| <b>NVIDIA Isaac Sim</b><br><a href="https://docs.isaacsim.omniverse.nvidia.com/6.0.0/installation/requirements.html">docs.isaacsim.omniverse.nvidia.com/6.0.0/installation/requirements.html</a> | Free download                                                                       | Photoreal simulation; needs at least an RTX 4080 with 16 GB VRAM |
| Simulator                                                                                                                                                                                        | When to use it                                                                      |                                                                  |
| <b>Gazebo</b>                                                                                                                                                                                    | Learn first; natively integrated with ROS 2 and runs on your laptop                 |                                                                  |
| <b>MuJoCo</b>                                                                                                                                                                                    | For reinforcement learning and locomotion; still no GPU needed                      |                                                                  |
| <b>Isaac Sim, Isaac Lab</b>                                                                                                                                                                      | Only with RTX hardware and a specific reason; A100 and H100 cards are not supported |                                                                  |
| <b>PyBullet</b>                                                                                                                                                                                  | Skip; no release since 2022 and the issue tracker is closed                         |                                                                  |
| <b>Genesis</b>                                                                                                                                                                                   | Worth watching, but v1.0 is new and has no ROS 2 integration                        |                                                                  |

**PRACTICE TASK**

Put your xacro robot into Gazebo, add lidar and camera plugins, and build a custom SDF world with obstacles. Confirm the sensor data appears on ROS 2 topics and in RViz.

## 4.5 ros2\_control

Few self-taught candidates have used `ros2_control`, so it is a fast way to stand out.

| Resource                                                                                                                                                                                                                                          | Cost | Use it for                                          |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|-----------------------------------------------------|
| <b>ros2_control documentation</b><br><a href="https://control.ros.org/rolling/index.html">control.ros.org/rolling/index.html</a>                                                                                                                  | Free | Hardware interfaces, controller manager, xacro tags |
| <b>Articulated Robotics: ros2_control on real hardware</b><br><a href="https://articulatedrobotics.xyz/tutorials/mobile-robot/applications/ros2_control-real/">articulatedrobotics.xyz/tutorials/mobile-robot/applications/ros2_control-real/</a> | Free | Moving from simulation to real hardware             |

**PRACTICE TASK**

Add `ros2_control` tags to your robot, configure `diff_drive_controller` and `joint_state_broadcaster` in YAML, and drive it in Gazebo with keyboard teleop. Then write an action server that drives a commanded distance and reports progress, with feedback and cancellation.

## 4.6 SLAM and navigation

| Resource                                                                                                                                               | Cost                                                                    | Use it for                                                      |
|--------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|-----------------------------------------------------------------|
| <b>Nav2 Getting Started</b><br><a href="https://docs.nav2.org/rolling/getting_started/index.html">docs.nav2.org/rolling/getting_started/index.html</a> | Free                                                                    | Nav2 in simulation within minutes, with a VS Code dev container |
| <b>Nav2 Tutorials</b><br><a href="https://docs.nav2.org/rolling/tutorials/">docs.nav2.org/rolling/tutorials/</a>                                       | Free                                                                    | SLAM, keepout zones, speed limits, docking, custom plugins      |
| <b>SLAM Toolbox</b><br><a href="https://github.com/SteveMacenski/slam_toolbox">github.com/SteveMacenski/slam_toolbox</a>                               | Free, open source                                                       | Supported 2D SLAM for ROS 2, including localization-only mode   |
| <b>RTAB-Map for ROS 2</b><br><a href="https://github.com/introlab/rtabmap_ros">github.com/introlab/rtabmap_ros</a>                                     | Free, open source                                                       | RGB-D and stereo SLAM with loop closure and 3D maps             |
| Key concept                                                                                                                                            | Explain                                                                 |                                                                 |
| <b>Mapping vs. localization</b>                                                                                                                        | When to switch SLAM Toolbox into localization mode                      |                                                                 |
| <b>Odometry quality</b>                                                                                                                                | A smeared map is usually an odometry problem, not an algorithm problem  |                                                                 |
| <b>Costmap tuning</b>                                                                                                                                  | Inflation radius, obstacle layers, and the local versus global costmaps |                                                                 |
| <b>Behavior trees</b>                                                                                                                                  | How Nav2 coordinates everything, and the part beginners most often miss |                                                                 |

| Tier           | Cost           | Hardware for real SLAM                                                                                                                                                                                                                                                                                                                        |
|----------------|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Simulation     | \$0            | Gazebo with a simulated lidar. Complete the full SLAM and Nav2 work here first.                                                                                                                                                                                                                                                               |
| DIY            | \$250 to \$450 | RPLIDAR C1 (\$69), Raspberry Pi 4 or 5, differential-drive base with encoders, motor driver, battery. Most learning per dollar.                                                                                                                                                                                                               |
| Ready platform | \$300 to \$535 | Hiwonder MentorPi M1 from \$299.99 ( <a href="http://www.hiwonder.com/products/mentorpi-m1">www.hiwonder.com/products/mentorpi-m1</a> );<br>Waveshare UGV Rover ROS 2 kit, \$534.99, with a Pi host and ESP32 real-time controller ( <a href="http://www.waveshare.com/ugv-rover-ros2-kit.htm">www.waveshare.com/ugv-rover-ros2-kit.htm</a> ) |

**PRACTICE TASK**

Run SLAM Toolbox in your Gazebo world, teleop around it, and save the map. Switch to localization mode and send Nav2 goals from RViz. Tune the costmaps until the robot stops cutting corners, then screen-record the result.

**MONTH 4 MILESTONE: BY THE END OF THIS MONTH YOU CAN**

- ☐ Write ROS 2 nodes in Python and C++ that use topics, services, and actions
- ☐ Describe your own robot in xacro with correct frames, inertias, and collision geometry
- ☐ Simulate it in Gazebo with working lidar and camera sensors
- ☐ Drive the robot through ros2\_control instead of raw commands
- ☐ Build a map with SLAM Toolbox and navigate autonomously with Nav2
- ☐ Diagnose a broken TF tree, the most common failure in the stack

MONTH

5

## The math that makes robots work

**Goal:** Understand and implement the control and perception underneath what you have built.

Knowing what the libraries do is what lets you fix them when they misbehave. Aim for working fluency in control, enough kinematics to reason about an arm, and enough vision to get 3D information from a camera.

### 5.1 Control theory, starting with PID

| Resource                                                                                                                                                                                                                                               | Cost      | Use it for                                       |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|--------------------------------------------------|
| <b>Understanding PID Control, MATLAB Tech Talks (Brian Douglas)</b><br><a href="http://www.mathworks.com/videos/series/understanding-pid-control.html">www.mathworks.com/videos/series/understanding-pid-control.html</a>                              | Free      | Seven parts: windup, filtering, tuning           |
| <b>Brian Douglas: Control System Lectures</b><br><a href="http://www.youtube.com/@BrianBDouglas/playlists">www.youtube.com/@BrianBDouglas/playlists</a>                                                                                                | Free      | Intuition-first videos from PID to state space   |
| <b>The Fundamentals of Control Theory, Brian Douglas</b><br><a href="http://engineeringmedia.com/books">engineeringmedia.com/books</a>                                                                                                                 | Free (CC) | Written companion to the videos                  |
| <b>Control Bootcamp, Steve Brunton</b><br><a href="http://www.youtube.com/playlist?list=PLMrJAKhIeNNR20Mz-VpzgQs5ZrYi085m">www.youtube.com/playlist?list=PLMrJAKhIeNNR20Mz-VpzgQs5ZrYi085m</a>                                                         | Free      | State space, controllability, LQR, Kalman filter |
| <b>Feedback Systems, Åström and Murray</b><br><a href="http://fbswiki.org/wiki/index.php/Feedback_Systems:_An_Introduction_for_Scientists_and_Engineers">fbswiki.org/wiki/index.php/Feedback_Systems:_An_Introduction_for_Scientists_and_Engineers</a> | Free PDF  | The rigorous textbook                            |

| Key concept               | Explain                                                        |
|---------------------------|----------------------------------------------------------------|
| <b>P, I, and D</b>        | What each term physically does and how each one fails          |
| <b>Integrator windup</b>  | Why an arm lurches when it comes off a limit                   |
| <b>Derivative noise</b>   | Why the D term amplifies sensor noise and needs filtering      |
| <b>Steady-state error</b> | When an integrator is the fix and when gravity compensation is |
| <b>Feedforward</b>        | A cheap performance gain that most people skip                 |

#### PRACTICE TASK

On the Month 2 balancing robot, implement three controllers: P only, PD, and PID with feedforward. Log each step response to CSV, plot all three together, and write up which one you would ship and why.

### 5.2 State space, LQR, and MPC

| Resource                                                                                                                                                                                                                                         | Cost | Use it for                                           |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|------------------------------------------------------|
| <b>Understanding Model Predictive Control, MATLAB Tech Talks</b><br><a href="http://www.mathworks.com/videos/series/understanding-model-predictive-control.html">www.mathworks.com/videos/series/understanding-model-predictive-control.html</a> | Free | From motivation to fast, nonlinear MPC               |
| <b>Underactuated Robotics, Russ Tedrake (MIT)</b><br><a href="http://underactuated.csail.mit.edu/index.html">underactuated.csail.mit.edu/index.html</a>                                                                                          | Free | Control applied to walking, balancing, and humanoids |

| Key concept        | Explain                                         |
|--------------------|-------------------------------------------------|
| <b>State space</b> | Describing a system by state, input, and output |

| Key concept    | Explain                                                                    |
|----------------|----------------------------------------------------------------------------|
| LQR            | A principled way to choose controller gains                                |
| MPC            | Handles constraints, at the cost of computation                            |
| Underactuation | Robots with fewer actuators than degrees of freedom need different methods |

**PRACTICE TASK**

Implement LQR for a simulated cart-pole in Python, then a hand-tuned PID for the same system, and compare how each handles a disturbance. Tedrake's notes supply the model.

## 5.3 Kinematics and dynamics

| Resource                                                                                                                                                                                | Cost                    | Use it for                                  |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------|---------------------------------------------|
| <b>Modern Robotics, Kevin Lynch (Northwestern)</b><br><a href="https://hades.mech.northwestern.edu/index.php/Modern_Robotics">hades.mech.northwestern.edu/index.php/Modern_Robotics</a> | Free book, code, videos | Standard text using screw theory            |
| <b>Modern Robotics Specialization (Coursera)</b><br><a href="https://www.coursera.org/specializations/modernrobotics">www.coursera.org/specializations/modernrobotics</a>               | Free to audit           | Same material with deadlines and assessment |
| <b>Robotics Toolbox for Python, Peter Corke</b><br><a href="https://github.com/petercorke/robotics-toolbox-python">github.com/petercorke/robotics-toolbox-python</a>                    | Free                    | FK, Jacobians, IK, and models of real arms  |
| <b>QUT Robot Academy, Peter Corke</b><br><a href="https://robotacademy.net.au">robotacademy.net.au</a>                                                                                  | Free                    | 200+ short video lessons by level           |

| Key concept                    | Explain                                                   |
|--------------------------------|-----------------------------------------------------------|
| Homogeneous transforms         | Composing transforms, the shared language of robotics     |
| Forward and inverse kinematics | FK is straightforward; IK is not                          |
| Jacobian and singularities     | Why an arm suddenly cannot move in some direction         |
| Workspace                      | Joint limits versus reachability                          |
| Trajectory generation          | When to interpolate in joint space versus Cartesian space |

**PRACTICE TASK**

Compute the forward kinematics of your SO-101 by hand from its link lengths and verify with the Robotics Toolbox. Then write a numerical IK solver that moves the end effector to a commanded XYZ position, and observe its behavior near a singularity.

## 5.4 Perception and computer vision

| Resource                                                                                                                                                                                   | Cost | Use it for                                                 |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|------------------------------------------------------------|
| <b>OpenCV Bootcamp (OpenCV.org)</b><br><a href="https://courses.opencv.org/courses/course-v1:OpenCV+Bootcamp+CV0/about">courses.opencv.org/courses/course-v1:OpenCV+Bootcamp+CV0/about</a> | Free | Official 2 to 3 hour introductory course                   |
| <b>OpenCV Camera Calibration tutorial</b><br><a href="https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html">docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html</a>         | Free | Chessboard calibration through reprojection error          |
| <b>Cyrril Stachniss lectures (University of Bonn)</b><br><a href="https://www.ipb.uni-bonn.de/online-training-robotics/">www.ipb.uni-bonn.de/online-training-robotics/</a>                 | Free | Geometry, bundle adjustment, EKF and graph SLAM            |
| <b>Open3D point cloud tutorials</b><br><a href="https://www.open3d.org/docs/release/tutorial/geometry/pointcloud.html">www.open3d.org/docs/release/tutorial/geometry/pointcloud.html</a>   | Free | Downsampling, normals, ICP, plane segmentation, clustering |

| Key concept        | Explain                                                                 |
|--------------------|-------------------------------------------------------------------------|
| Camera intrinsics  | Distortion and a real chessboard calibration                            |
| Pinhole projection | Image coordinates versus world coordinates                              |
| Depth sensing      | Stereo versus structured light versus time-of-flight                    |
| Point clouds       | Downsampling, fitting a plane to find a table, clustering objects on it |
| Lighting           | Why a vision pipeline that worked yesterday can fail today              |

**PRACTICE TASK**

Calibrate a real camera with a printed chessboard and save the intrinsics. Write a script that detects a colored object and estimates its 3D position relative to the camera. Change the lighting, watch it fail, fix it, and write up the failure and the fix.

**5.5 Manipulation and MoveIt 2**

| Resource                                                                                                                                                                                                                                                                                                                                        | Cost      | Use it for                                                 |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|------------------------------------------------------------|
| <b>MoveIt 2 Getting Started</b><br><a href="https://moveit.picknik.ai/main/doc/tutorials/getting_started/getting_started.html">moveit.picknik.ai/main/doc/tutorials/getting_started/getting_started.html</a>                                                                                                                                    | Free      | Official entry point; Jazzy on Ubuntu 24.04 is recommended |
| <b>Pick and Place with MoveIt Task Constructor</b><br><a href="https://moveit.picknik.ai/main/doc/tutorials/pick_and_place_with_moveit_task_constructor/pick_and_place_with_moveit_task_constructor.html">moveit.picknik.ai/main/doc/tutorials/pick_and_place_with_moveit_task_constructor/pick_and_place_with_moveit_task_constructor.html</a> | Free      | Breaking a task into stages: grasps, IK, collisions        |
| <b>Robotic Manipulation, Russ Tedrake (MIT)</b><br><a href="https://manipulation.csail.mit.edu/">manipulation.csail.mit.edu/</a>                                                                                                                                                                                                                | Free      | Full manipulation stack, model-based and learned           |
| <b>Contact-GraspNet (NVIDIA)</b><br><a href="https://github.com/NVlabs/contact_graspnet">github.com/NVlabs/contact_graspnet</a>                                                                                                                                                                                                                 | Free code | Baseline for learned 6-DOF grasping in clutter             |

**PRACTICE TASK**

Get MoveIt 2 planning for a simulated arm, add collision objects to the planning scene, and run a pick and place. Then block the only viable path and observe how the planner responds. Knowing how planners fail is worth more than watching them succeed.

**MONTH 5 MILESTONE: BY THE END OF THIS MONTH YOU CAN**

- ☐ Tune a PID controller and explain every term using measured data
- ☐ Model a system in state space and apply LQR to a simulated plant
- ☐ Compute forward kinematics by hand and solve IK numerically
- ☐ Show a singularity on a real or simulated arm
- ☐ Calibrate a camera and convert a pixel into a 3D position
- ☐ Plan and execute a collision-free pick and place in MoveIt 2

MONTH

6

## Robot learning, specialization, and getting hired

**Goal:** Choose a direction, finish a portfolio piece in it, and start applying.

The first half of the month covers teaching robots from demonstrations, the area of robotics that has changed most in recent years. The second half turns your work into a portfolio that gets interviews.

### 6.1 LeRobot

LeRobot is Hugging Face's PyTorch library for real-world robot learning. The workflow is teleoperate, record, train, and deploy: you drive the robot by hand, demonstrations are saved as synchronized video and actions, a policy learns to imitate them, and the robot then runs on its own.

| Resource                                                                                                                                               | Cost              | Use it for                                                   |
|--------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|--------------------------------------------------------------|
| <b>LeRobot documentation</b><br><a href="https://huggingface.co/docs/lerobot/index">huggingface.co/docs/lerobot/index</a>                              | Free              | Full pipeline and all supported robots                       |
| <b>LeRobot repository</b><br><a href="https://github.com/huggingface/lerobot">github.com/huggingface/lerobot</a>                                       | Free, Apache 2.0  | Read how the policies are implemented                        |
| <b>Hugging Face Robotics Course</b><br><a href="https://huggingface.co/learn/robotics-course/unit0/1">huggingface.co/learn/robotics-course/unit0/1</a> | Free, no hardware | Simulated environments and public datasets                   |
| <b>SO-101 setup guide</b><br><a href="https://huggingface.co/docs/lerobot/so101">huggingface.co/docs/lerobot/so101</a>                                 | Free              | Port setup, motor setup, calibration, and recording commands |

| Key concept                  | Explain                                                                                                 |
|------------------------------|---------------------------------------------------------------------------------------------------------|
| <b>Record, train, deploy</b> | The full loop, end to end, on your own arm                                                              |
| <b>Dataset quality</b>       | Sloppy demonstrations produce a sloppy policy                                                           |
| <b>ACT</b>                   | The recommended starting policy; predicts chunks of future actions instead of single steps              |
| <b>Diffusion Policy</b>      | Treats action generation as denoising; reported a 46.9% average gain over prior methods across 12 tasks |

#### PRACTICE TASK

Record 50 demonstrations of one simple task on your SO-101, such as moving a cube into a bin. Train an ACT policy and deploy it; expect it to succeed about half the time. Record 50 more demonstrations that cover the failure cases, retrain, and report the success rate before and after. The measured improvement is the portfolio piece.

### 6.2 Vision-language-action (VLA) models

| Model                                                                                                                                                                 | Openness                                                 | Notes                                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|---------------------------------------------------------------------------------------|
| $\pi_0$ , $\pi_0$ -FAST, $\pi_{0.5}$ (Physical Intelligence)<br><a href="https://github.com/Physical-Intelligence/openpi">github.com/Physical-Intelligence/openpi</a> | Apache 2.0, open weights                                 | Pretrained on 10,000+ hours of robot data; transfer to other robots is not guaranteed |
| <b>OpenVLA</b><br><a href="https://openvla.github.io/">openvla.github.io/</a>                                                                                         | Fully open                                               | 7B parameters, trained on 970,000 Open X-Embodiment episodes; well documented         |
| <b>GR00T N1.7</b> (NVIDIA)<br><a href="https://github.com/Nvidia/Isaac-GR00T">github.com/Nvidia/Isaac-GR00T</a>                                                       | Code Apache 2.0; weights under NVIDIA Open Model License | Needs 16 GB+ VRAM for inference                                                       |
| <b>SmolVLA</b> (Hugging Face)<br><a href="https://huggingface.co/lerobot/smolvla_base">huggingface.co/lerobot/smolvla_base</a>                                        | Open                                                     | Compact; the right choice for fine-tuning on an SO-101                                |
| <b>RT-2</b> (Google DeepMind)                                                                                                                                         | No public weights                                        | Historically important; study the paper                                               |

## 6.3 Reinforcement learning for robotics

| Resource                                                                                                                                                                | Cost             | Use it for                                                   |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|--------------------------------------------------------------|
| <b>MuJoCo Playground</b><br><a href="https://github.com/google-deepmind/mujoco_playground">github.com/google-deepmind/mujoco_playground</a>                             | Free, Apache 2.0 | GPU environments with Colab tutorials; start here            |
| <b>NVIDIA Isaac Lab</b><br><a href="https://github.com/isaac-sim/IsaacLab">github.com/isaac-sim/IsaacLab</a>                                                            | Free, BSD-3      | Industry standard for legged sim-to-real; needs RTX hardware |
| <b>CS 285 Deep Reinforcement Learning, Sergey Levine (UC Berkeley)</b><br><a href="https://rail.eecs.berkeley.edu/deeprlcourse">rail.eecs.berkeley.edu/deeprlcourse</a> | Free             | Imitation learning through offline RL                        |

### PRACTICE TASK

Train a quadruped locomotion policy using a MuJoCo Playground Colab tutorial, then change the reward function and observe how the gait changes. No hardware or paid GPU is needed.

## 6.4 Pick a direction

Choose one of three directions and keep the other two at a working-knowledge level.

| Direction                                      | Best if you want                                                      | Focus on                                                                                |
|------------------------------------------------|-----------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <b>Robot learning and embodied AI</b>          | Frontier companies and the highest ceiling; also the most competitive | LeRobot, VLA fine-tuning, imitation learning, RL, simulation, PyTorch                   |
| <b>Autonomy and mobile robotics</b>            | The largest number of openings                                        | ROS 2, Nav2, SLAM, perception, sensor fusion, C++                                       |
| <b>Embedded, mechatronics, and integration</b> | Work soon, including as a contractor                                  | Firmware, motor control, real-time systems, PLCs, functional safety, system integration |

A dataset you collected yourself carries weight in the robot learning track. In the integration track, functional safety is an underserved specialty, since certification is a common cause of project delays.

## 6.5 Your portfolio

| Strong signals                                                                         | Red flags                                                           |
|----------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| GitHub repos with logged metrics and a commit history that shows iterative debugging   | Still on end-of-life ROS 1 with no sign of migration                |
| Real-robot deployment with reliability data, not simulation only                       | Projects that fall apart under follow-up questions                  |
| Public datasets on the LeRobot Hub                                                     | Deep learning background with no kinematics or embodiment knowledge |
| Contributions to ROS 2, Nav2, MoveIt, Isaac Lab, or LeRobot                            | Tutorial completion certificates offered in place of real work      |
| Integration projects linking sensors, planning, and control, with video and clear docs |                                                                     |

### PRACTICE TASK

Rewrite the READMEs of your three best projects. Each needs a video at the top, a wiring or architecture diagram, the numbers you measured, and a section titled "What broke and how I fixed it." That section is the hardest part of a portfolio to fake.

## 6.6 Interviews

Robotics interviews differ from software interviews, and algorithm puzzles predict much less. Expect questions on inverse kinematics, PID and feedback, sensor fusion and Kalman filters, SLAM, path planning including RRT, and C++ versus Python trade-offs. Strong companies may also give you a simulation scene with a deliberately broken controller to debug, a ROS 2 architecture problem tied to the role, and a long discussion of a past failure. Real examples:

[www.glassdoor.com/Interview/robotics-engineer-interview-questions-SRCH\\_K00,17.htm](http://www.glassdoor.com/Interview/robotics-engineer-interview-questions-SRCH_K00,17.htm)

### PRACTICE TASK

Have someone question you about your own repository for twenty minutes, focused on your specific code rather than general concepts: why that gain, why that sensor, what happens when the battery sags, what you tried first. If you cannot answer three levels deep, the project is not ready for your CV.

### MONTH 6 MILESTONE: BY THE END OF THIS MONTH YOU CAN

- ☐ Record a demonstration dataset and train a policy that runs on your own hardware
- ☐ Explain the differences between behavior cloning, ACT, and diffusion policy
- ☐ Name which VLA models have open weights and which do not
- ☐ State which direction you are pursuing and why
- ☐ Show three portfolio projects with video, metrics, and a documented failure analysis
- ☐ Answer three levels of follow-up questions about any part of your own code

## Career outlook

The original article ends with a realistic view of the market. Six months of focused work will not make you a senior engineer, a level that usually means five or more years of shipping real-time systems on physical robots. It can make you someone who builds, debugs, and ships working robots across the full stack, with a portfolio to prove it, and that is enough to enter the field.

Investment is strong: Crunchbase reported \$47.4B raised by physical AI startups in the first half of 2026. Hiring is growing more slowly than funding. A3 reported 2.0% unit growth in North American robot orders over the same period, and the BLS projects 1 to 2% growth for the occupation group that includes robotics engineers. Demand is concentrated in specific specialties, and entry roles such as robotics technician are open to candidates without a four-year degree.

| Role or segment (US unless noted)        | Reported pay                                                               | Source or context                     |
|------------------------------------------|----------------------------------------------------------------------------|---------------------------------------|
| <b>Robotics engineer, median</b>         | \$122,930                                                                  | O*NET / BLS                           |
| <b>Entry level</b>                       | About \$80k to \$100k                                                      | Payscale, Salary.com                  |
| <b>Mid level, 3 to 6 years</b>           | \$120k to \$165k                                                           | Conventional employers                |
| <b>Senior</b>                            | \$160k to \$230k                                                           | Mainstream employers                  |
| <b>Frontier physical-AI roles</b>        | \$200k to \$400k base                                                      | Live Figure job listings              |
| <b>Robotics technician, median</b>       | \$73,900                                                                   | BLS                                   |
| <b>Teleoperation and data collection</b> | About \$28/hour average                                                    | Job postings; mostly high-cost metros |
| <b>Freelance ROS work</b>                | \$22 to \$70/hour; about \$500 and up per node                             | Upwork                                |
| <b>Outside the US</b>                    | Germany ~€70k; Switzerland ~CHF 95k; UK £32k to £50k base; India ~INR 653k | Glassdoor and others                  |

One more point from the original: in an industrial robot cell, the robot itself is only about a quarter of the total project cost. The rest is engineering, safety, and integration, which is why integration skills are a career in their own right.

## Three habits to keep

| Habit                              | Explain                                                                                                                                                                                                  |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Build, do not just study</b>    | Complete at least one build each month: the line follower, the balancer, the arm, the navigation stack, the trained policy. Break it, fix it, film it, and push it to GitHub.                            |
| <b>Write down what broke</b>       | Record the failures you hit first and how you diagnosed each one. That record is the strongest evidence of engineering ability, and it is what lets a project hold up in an interview.                   |
| <b>Start before you feel ready</b> | Apply for technician or teleoperation roles, contribute a small fix to an open-source package such as Nav2, or reach out to a small robotics startup. Nobody will tell you that you have studied enough. |

### CREDIT

Original article: "How to become a Robotics Engineer in 6 months (RESOURCES)" by Ronin (@deronin\_) on X, September 2, 2026.

[https://x.com/deronin\\_/status/2095180126359105955](https://x.com/deronin_/status/2095180126359105955)

This handout is an adapted summary for students. All credit for the roadmap and resource research belongs to the original author.